PROJECT REPORT

Maze Solver using BFS and DFS

---

1. Introduction

The major focus in Artificial Intelligence (AI) is on developing intelligent systems that can solve complex problems in an efficient manner. One of the major techniques used in AI is search algorithms, used to solve problems in an intelligent manner.

The project will show how two major search algorithms are used to solve problems in an intelligent manner:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)

The search algorithms will be used to solve a maze pathfinding problem.

---

2. Problem Statement

Finding the optimal path from a source to a destination within a constrained environment is a common problem in real-world applications, and some of the applications include:

- Navigation systems
- Robotics path planning
- Game development
- Network routing

However, it is very difficult to grasp how different algorithms solve this problem without proper visualization.

---

3. Objective

The main objectives of this project are:

- To implement BFS and DFS algorithms
- To solve a maze using these algorithms
- To compare their performance visually and quantitatively

- To demonstrate differences between optimal and non-optimal solutions

---

4. Methodology

4.1 Maze Representation

The maze is represented as a **2D grid**:

- 0 → Free path
- 1 → Wall/Obstacle

Example:

0 0 0

1 1 0

0 0 0

4.2 Random Maze Generation

A random maze is generated using a probability-based approach where:

- Each cell has a 30% chance of being a wall
- Start and goal positions are always kept open

The program ensures that a valid path exists by regenerating the maze until BFS finds a solution.

4.3 Breadth-First Search (BFS)

- Uses a **queue (FIFO)**
- Explores nodes level by level
- Guarantees the **shortest path** in an unweighted graph

Implementation uses:

queue = deque([start])

BFS stops when the goal is reached and reconstructs the path using a parent mapping.

4.4 Depth-First Search (DFS)

- Uses a **stack (LIFO)**
- Explores as deep as possible before backtracking
- Does **not guarantee shortest path**

DFS behavior is controlled using:

stack = [start]

The traversal order affects the path found.

4.5 Path Reconstruction

Both algorithms use a **parent dictionary** to reconstruct the path from goal to start:

parent[neighbor] = node

This ensures the final path can be traced correctly.

4.6 Visualization

The solution is visualized using Python libraries:

- Matplotlib
- NumPy

Features:

- Side-by-side comparison of BFS and DFS
- Step-by-step animation
- Color-coded representation:
    - Path → Green
    - Walls → Dark
    - Start → Red
    - Goal → Blue

Visualization function:

---

5. Results and Observations

Key Observations:

- BFS always finds the **shortest path**
- DFS may find a **longer or different path**
- BFS explores nodes level-wise
- DFS explores nodes depth-wise

Example Output:

- BFS Path Length: Shorter
- DFS Path Length: Longer

This confirms that BFS is optimal for shortest path problems in unweighted graphs.

---

6. Comparison between BFS and DFS

Feature	BFS	DFS
Data Structure	Queue	Stack
Path Type	Shortest	Not guaranteed
Exploration	Level-wise	Depth-wise
Optimality	Yes	No
Performance	More memory	Less memory

---

7. Real-World Applications

The maze problem is a simplified representation of real-world pathfinding problems:

- Navigation systems (e.g., shortest route finding)
- Robotics (movement planning)
- Game AI (NPC pathfinding)
- Network routing (data packet traversal)

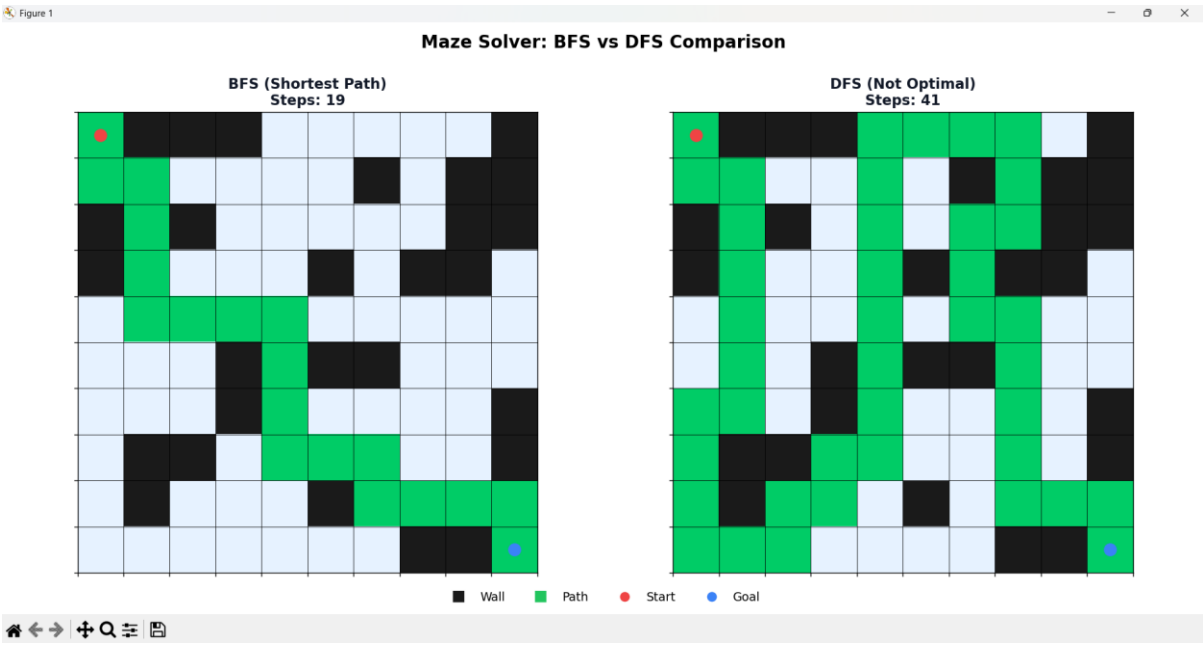
---

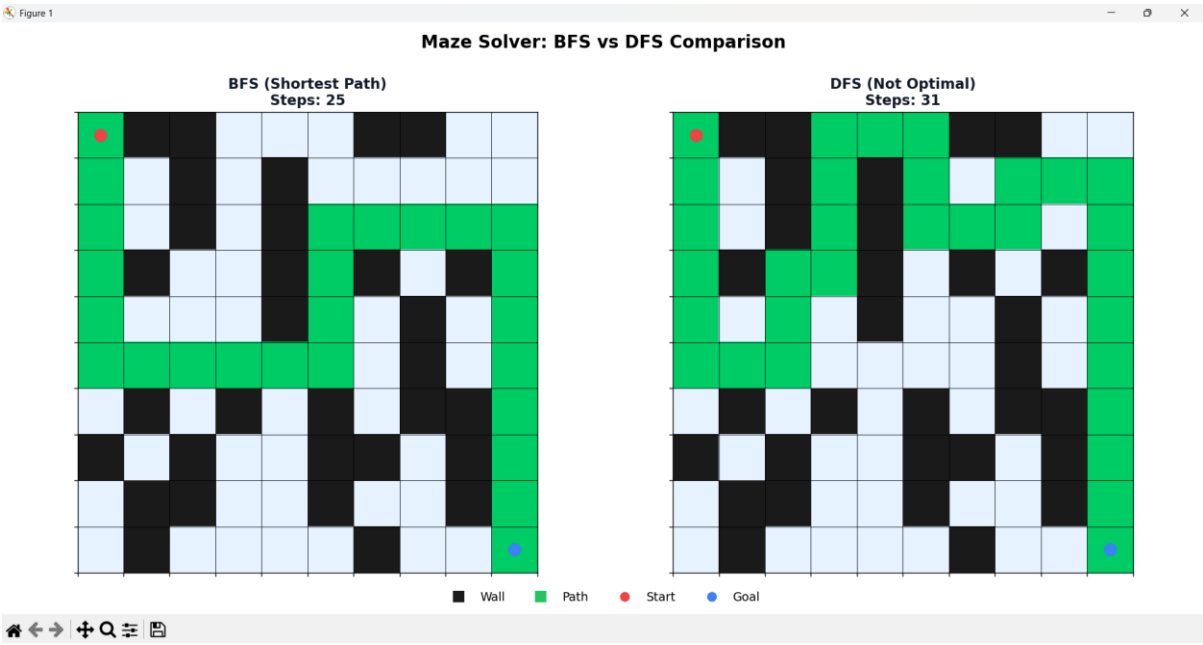
8. Challenges Faced

- Ensuring generated maze has a valid path

- Handling DFS traversal order differences
- Designing a clear and understandable visualization
- Synchronizing BFS and DFS animations

9.Results





---

10. Learning Outcomes

Through this project, the following concepts were learned:

- Implementation of AI search algorithms
- Differences between BFS and DFS
- Graph traversal techniques
- Visualization of algorithms
- Problem-solving using AI concepts

---

11. Conclusion

This project successfully demonstrates how BFS and DFS algorithms can be applied to solve pathfinding problems. The side-by-side visualization clearly highlights the differences between the two approaches.

BFS proves to be optimal for shortest path problems, while DFS provides an alternative exploration strategy. The project effectively bridges theoretical AI concepts with practical implementation.

---

12. References

- Artificial Intelligence: A Modern Approach – Stuart Russell & Peter Norvig
- Course Lecture Notes
- Python Documentation (Matplotlib, NumPy)